

How the error actually gets emitted

Before you write a single lifecycle method, it helps to know why a normal try/catch never even sees the crash.

- 1 Your **try/catch** only wraps code you call yourself. React invokes your components internally, inside its own render phase.
- 2 When a child throws while rendering, React's **fiber reconciler** catches that error first, not your code.
- 3 React then walks **up the fiber tree**, looking for the nearest ancestor that defines the error boundary lifecycle methods.
- 4 That upward walk is the emission. Only a parent class component with those methods can intercept it, never the component that threw.

A try/catch around your JSX does nothing here. The throw happens outside your function's normal call stack, deep inside React's own render cycle.

The algorithm, boiled down

Six steps, from the moment a child throws to the moment it's logged.

`<ErrorBoundary>`

01 · RENDER PHASE

Child component throws while rendering

02 · INTERNAL

React's fiber reconciler catches it internally

03 · PROPAGATION

Walks up the tree to the nearest ErrorBoundary

04 · STATE PATCH

`getDerivedStateFromError(error) → hasError: true`

05 · COMMIT

React commits: fallback UI is painted

06 · SIDE EFFECT, AFTER COMMIT

`componentDidCatch(error, info) → log to Sentry`

Give the boundary something to remember

constructor()

● ● ● ErrorBoundary.jsx

```
class ErrorBoundary extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { hasError: false };  
  }  
}
```

- > One boolean is enough. **hasError** decides which tree renders next, nothing more.
- > This state lives inside the boundary itself, completely separate from whatever crashed below it.
- > It starts false, so children render exactly as normal until something actually breaks.

Catch it before the next paint

getDerivedStateFromError()

● ● ● ErrorBoundary.jsx

```
static getDerivedStateFromError(error) {  
  return { hasError: true };  
}
```

- > A static method, so there's no **this**. It can't touch instance properties or fire side effects.
- > React calls it during the render phase itself, the moment a descendant throws.
- > Its only job is returning a state patch. That patch is what flips the fallback on for the next render.

Log it. Don't render it.

`componentDidCatch()``● ● ● ErrorBoundary.jsx`

```
componentDidCatch(error, info) {  
  logErrorToService(error, info.componentStack);  
}
```

- > Runs in the commit phase, after the DOM already reflects the fallback state.
- > This is where side effects belong: Sentry, Datadog, or your own logging pipeline.
- > **info.componentStack** tells you exactly which component tree produced the crash.

One flag decides everything

`render()``ErrorBoundary.jsx`

```
render() {  
  if (this.state.hasError) {  
    return <FallbackUI />;  
  }  
  return this.props.children;  
}
```

- > Checks **hasError** and picks one of two trees. That's the entire logic.
- > The boundary guards its children, not itself, not event handlers, not async code.
- > That's the boundary's actual boundary, and it's the part most candidates forget to say out loud.